

Lab2 实验报告

实验思路

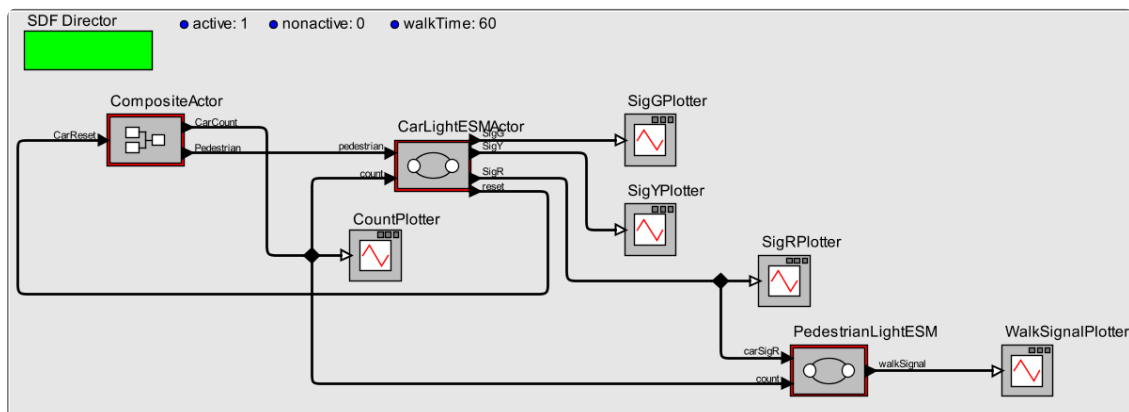
实验要求实现一个汽车-行人控制系统，在逻辑上，关键设计点如下：

- 整个系统共享一个计时变量 **count**，用以计时汽车交通灯绿灯的时间，同时能够给行人交通灯计算保持红灯的时间。
- 汽车交通灯的控制逻辑参考书 [LeeSeShia_DigitalV2_3](#) 中的 [Example3.9](#) 实现一个ESM，并输出 **SigG,SigY,SigR** 三个颜色的交通信号。
- 行人交通灯则基于汽车交通灯给出的红灯信号与共用的 **count** 计时变量控制交通灯的输出信号 **WalkSignal**，要么可通行要么不可通行。

Ptolemy实现

顶层模块

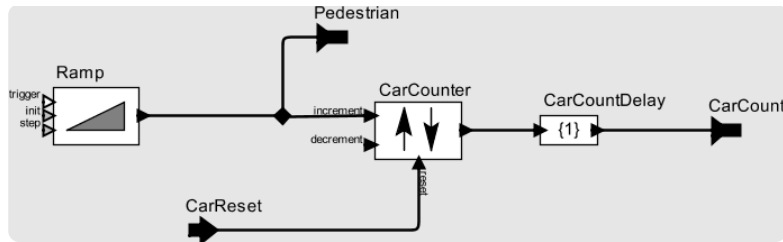
用 **SDF Director** 实现整个模型，顶层模块如下所示：



- **CompositeActor** 用以产生事件流输入信号 **Pedestrian** 与计时变量 **CarCount**，接受的 **CarReset** 将重置内部的 **Counter**。
- **CarLightESM** 是汽车交通灯控制模块，接受信号源产生的 **Pedestrian** 与全局共有的 **count** 计时变量，除输出 **SigG** 等三个代表交通灯的信号外，还会输出重置 **count** 的 **reset** 信号。
- **PedestrianLightESM** 是行人交通灯控制模块，接受汽车交通灯的红灯信号 **carSigR** 与计时变量 **count** 作为输入，输出 **walkSignal** 为指引行人通行的信号。
- 参数设置了三个：
 - **active** 代表对应交通灯处于激活状态，**nonactive** 反之。
 - **walkTime** 代表行人交通灯激活后可行走的时间，同样代表汽车交通灯单次汽车通行的最大时间。

CompositeActor

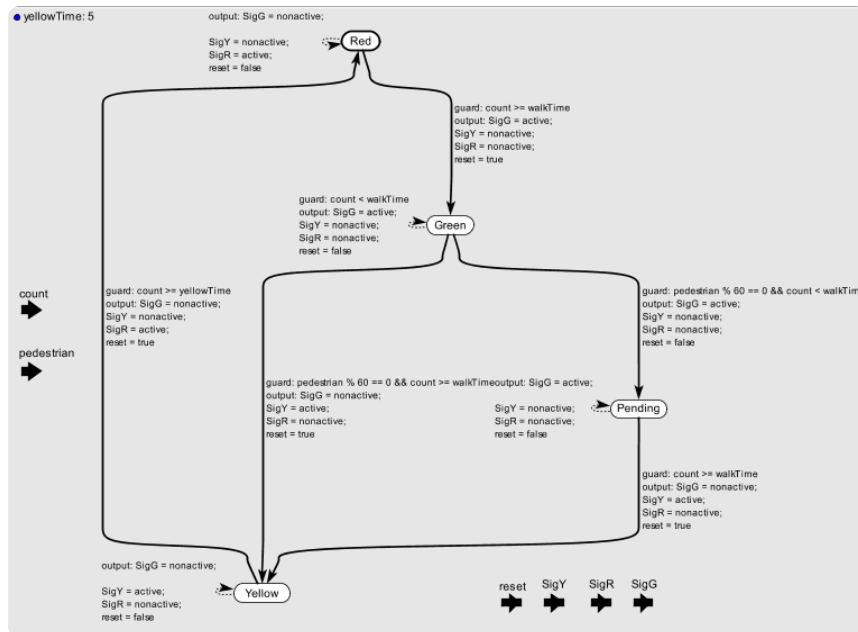
信号产生模块如下设计：



- 使用 **Ramp** 作为信号源，**step** 设为1，其输出不仅是信号源，也作为汽车交通灯的 **Pedestrian** 输入，其具体使用在下面的 **CarLightESM** 节中说明。
- 使用 **Counter** 作为全局计时变量 **count** 的产生器，**SampleDelay** 为所需的延迟。

CarLightESM

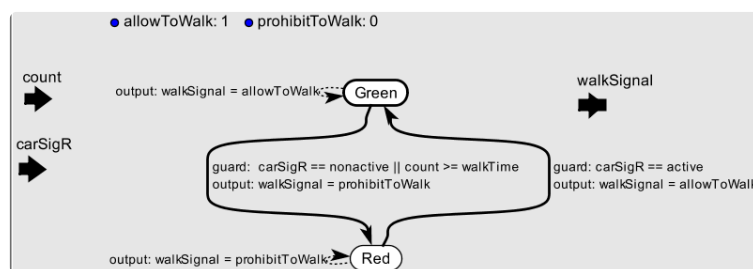
汽车交通灯控制模块如下：



其大致参考了书中例子，没有什么特殊的地方，主要是对输入的 **Pedestrian** 信号做对60取余的操作模拟每60个时间单位使能一次 **Pedestrian**。

PedestrianESM

行人交通灯模块设计如下：



- 初始状态为 **Green**，对应汽车交通灯的初始状态为 **Red**。
- Green** 到 **Red** 的转换条件要么是红灯不亮，要么就是允许通行时间到了需要转换到 **Red**
- Red** 到 **Green** 的转换条件与 **count** 没有关系，有且仅有汽车交通灯的红灯亮了，对应汽车交通灯由绿变红的转换只由外部输入 **Pedestrian** 影响而与 **count** 无关。

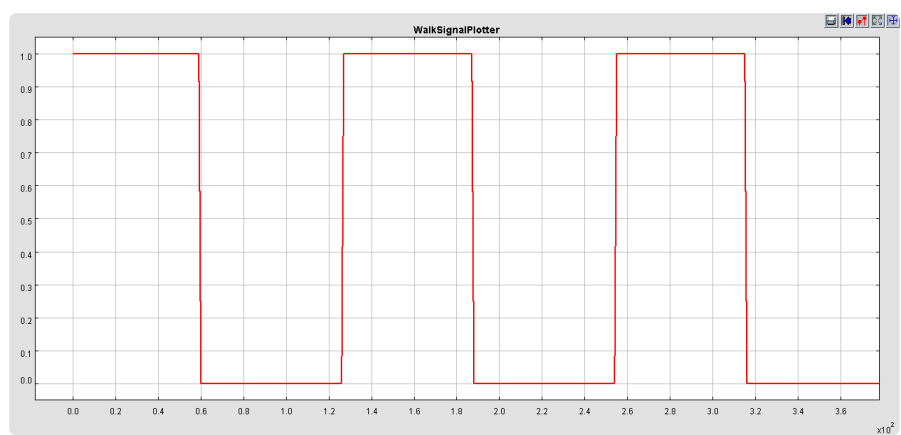
仿真与验证

实现后，它们的 **Plotter** 图如下所示，横轴范围均为 **0-360**，纵轴则是 **0-1**。

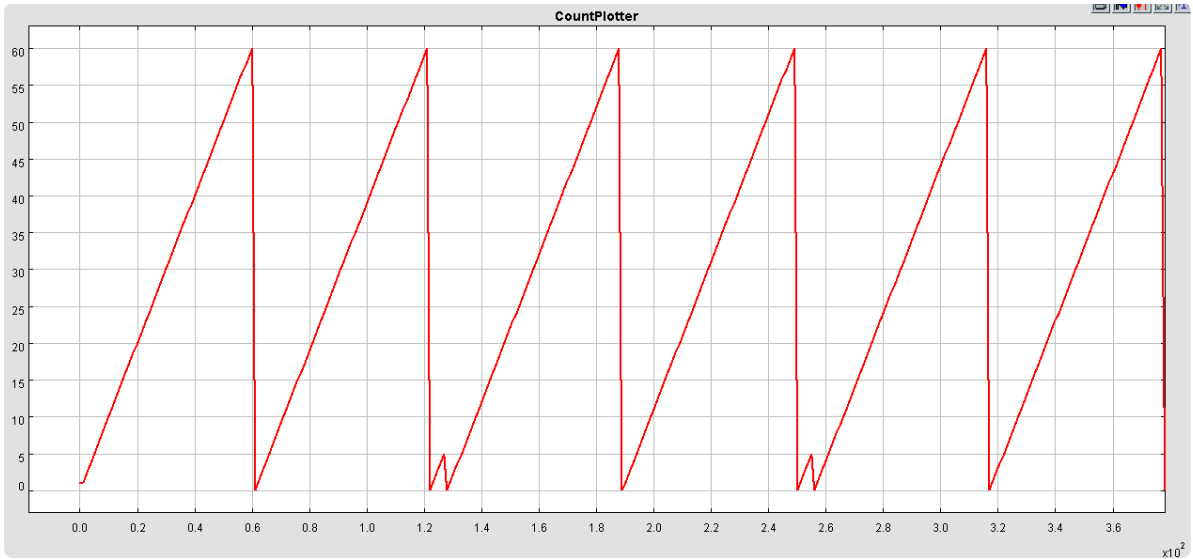


可以看到，在一开始，**SigR** 信号被拉高，在达到了禁止通行时间的上限60个时间单位后拉低，直接转换到了 **Green** 状态让汽车通行。

在第120个时间单位以前状态机都在 **Pending** 状态，所以保持 **SigG** 让汽车通行，而在第120个时间单位达到通行时间上限，从而让状态机转换到 **Yellow** 状态，此时 **SigY** 信号被拉高5个时间单位，而后转到了 **SigR** 并保持60个时间单位，后面就是这一部分的循环。



对应的，在 **SigR** 被拉高的时间点，**WalkSignal** 被拉高，代表此时允许通行，而一旦到了 **60**，就不再允许通行了，在 **120** 的时候没有直接拉高，因为这个时候汽车控制灯亮黄灯，直到汽车控制灯亮红灯才可通行。



`count` 的变化趋势也如图所示，在需要变黄灯的时候计时 5 而后再次拉低，整体符合逻辑。

问题分析

整个汽车与行人交通灯控制系统由两个有限状态机和一个组合协调模块构成，系统通过信号交互与同步调度实现汽车与行人交替通行的逻辑。

汽车模型通常包含三个基本状态：绿灯、黄灯和红灯。系统初始时汽车处于红灯状态，表示车辆禁止通行，红灯保持期间，系统会向行人模型发出允许通行的信号。当时间信号达到设定阈值后，状态从红灯切换为绿灯，用以提示汽车可通行。当时间信号再次达到对应设定阈值并且有人到来按下过街需求按钮（即上文的 `Pedestrian`）后，就会从绿灯转为黄灯提示即将禁止通行，黄灯维持一段较短的时间后，状态转为红灯，此时车辆禁止通行。随后在限制时间结束后，汽车重新进入绿灯状态，完成一个循环。

行人模型一般包含禁止通行、通行两个状态。当行人按钮被按下且汽车红灯亮起时，行人信号机由禁止通行状态进入通行状态，允许行人过马路。通行时间到达后，行人灯转为禁止通行状态，整个行人信号周期结束。

汽车与行人两个状态机并非独立运行，而是通过信号互锁与 `GuardCondition` 协调工作。汽车的红灯状态信号是行人灯切换为通行的必要条件；而行人通行周期结束的信号又是汽车重新点亮绿灯的触发条件。通过这种互锁机制，系统能够确保车辆与行人信号互不冲突。

组合模型（即 `CompositeActor`）承担了系统时序管理与信号协调的作用。它包含全局时钟或计时器用于提供时间节拍，`SampleDelay` 模块用于解决反馈环路带来的调度问题，使得模型在 SDF 领域下可被正确调度执行。各信号输出最终送入 `Plotter` 进行可视化显示。

在系统运行过程中，输入信号主要包括时间信号和行人请求信号；输出信号则反映当前交通灯的状态。模型的正确运行依赖于时序同步、信号依赖关系和状态机逻辑的正确设计。若计时周期设置不合理、Guard 条件错误或反馈延迟缺失，可能会导致调度失败、信号错乱或系统死锁。

总体来看，该系统通过有限状态机描述交通灯逻辑，通过信号耦合实现汽车与行人的互斥通行，通过时间驱动实现周期性切换。关键影响因素包括计时参数、状态机逻辑、信号同步和调度机制，它们共同决定了整个系统的稳定性与正确性。